Nationwide TDP 2026 Capstone Project — V2

Sprint 1 — Foundations, Data Model, API Skeleton & DevSecOps

Duration: 3 Days

Sprint Objective

Establish a reproducible technical and DevSecOps foundation so the team can demonstrate an initial end-to-end flow using seeded data, a working API skeleton and an automated security-aware development workflow.

The backend may be implemented using **C# or Python 3.14**.

The selected language should be used consistently for the API and unit tests.

Sprint 1 establishes the security pipeline that will be extended throughout the remaining sprints.

1. Backend Technology Choice

Option A — C#

Use C# for:

- API
- Business logic
- Unit tests

Option B — Python 3.14

Use Python 3.14 for:

- API
- Business logic
- Unit tests

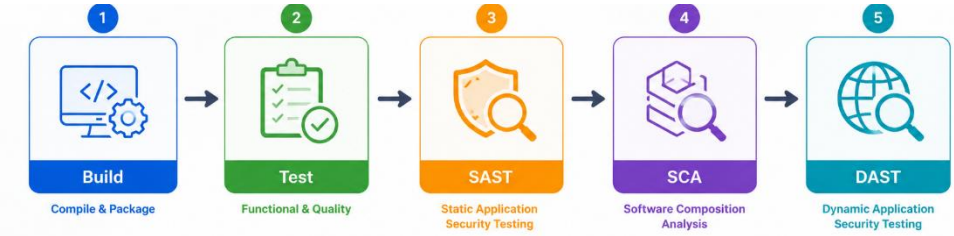
The language choice does not change:

- API contract
- Database model
- Security requirements
- Acceptance criteria

2. DevSecOps Workflow

Security should be integrated into development rather than treated as a final-stage activity.

Target Pipeline



Sprint 1 minimum requirement:

- Source control
- Automated build
- Automated unit tests
- SAST
- SCA
- Initial DAST against the API skeleton

The pipeline can initially run against the API skeleton rather than the completed carbon calculation engine.

3. Repository and Agile Setup

Create a GitHub repository for the project.

Recommended branching model:

main

dev

feature/*

bugfix/*

main and dev should be protected where possible, with changes introduced through pull requests and build validation.

Create:

- README
- Product backlog
- Sprint 1 issues
- Definition of Ready
- Definition of Done
- Contribution guidance

4. Project Structure

A suitable simplified structure is:

```
Nationwide-TDP-2026/
│
├── docs/
├── src/
│   └── api/
├── tests/
├── infra/
│   └── sql/
├── .github/
│   └── workflows/
└── README.md
```

The exact structure can be adapted by the team.

The provided QA starter skeleton may be used as the starting point.

5. SQL Data Model

Use **T-SQL** for the database.

Minimum entities:

- Categories
- EmissionsFactors
- Assumptions

Emissions factors should include:

- Source
- Version
- Effective date

Seed the database with synthetic data covering:

- Travel
- Food
- Utilities

Required files:

infra/sql/schema.sql

infra/sql/seed.sql

6. API Skeleton

Use the selected backend technology:

- C#
- Python 3.14

Create the initial endpoint:

POST /calculate

Sprint 1 does not require the complete carbon calculation engine.

The endpoint may initially return mock JSON:

```
{
  "status": "success",
  "totalCarbonKg": 0,
  "breakdown": [],
  "recommendations": []
}
```

Document:

- Request structure
- Response structure

- Expected HTTP status codes
- Basic validation requirements

7. Unit-Test Foundation

Create automated unit tests during Sprint 1.

Initial tests should confirm:

- API builds successfully
- API starts successfully
- Valid request can be submitted
- HTTP 200 is returned
- Response structure is correct
- Basic input validation works

Detailed carbon-calculation tests are expanded in Sprint 2.

8. SAST — Static Application Security Testing

SAST is incorporated directly into the DevSecOps pipeline.

The selected SAST tool should scan the application source code.

Example:

Build → Unit Tests → SAST

Document:

- SAST tool selected
- Code/components scanned
- Findings produced
- Severity levels
- What findings block the pipeline

Teams should demonstrate how a security finding is identified and remediated.

9. SCA — Software Composition Analysis

SCA should scan third-party dependencies.

The team should be able to identify:

- Dependency

- Version
- Vulnerability
- Severity
- Recommended remediation

Avoid introducing unnecessary third-party dependencies.

SCA results should form part of the DevSecOps evidence.

10. DAST — Dynamic Application Security Testing

DAST tests the **running application**, rather than simply analysing its source code.

Sprint 1 should perform an initial DAST scan against:

POST /calculate

Test areas should include:

- Invalid input
- Unexpected parameters
- Malformed requests
- Error handling
- Exposed technical errors
- Common web/API security weaknesses

Sprint 1 DAST Target

The purpose is to demonstrate security testing against a running API.

A fully implemented carbon engine is **not required** for the Sprint 1 DAST activity.

11. DevSecOps Evidence

Retain evidence of:

- Successful build
- Unit-test results
- SAST results
- SCA results
- DAST results
- Deliberately generated security finding

- Remediation of the finding

This evidence should be available for the final project demonstration.

12. Infrastructure Foundation

Establish a reproducible infrastructure approach.

The team should either provide:

Option A — Runbook

Document how to provision:

- SQL database
- API
- Required configuration

Option B — Infrastructure as Code

Create an initial Terraform structure:

infra/

└── terraform/

Full Terraform implementation can remain a stretch objective if time is limited.

13. Security Baseline

The following security practices apply from Sprint 1:

- No secrets committed to GitHub
- No real customer information
- Synthetic/anonymised data only
- Configuration separated from source code
- Dependencies identified
- Security scanning enabled

Azure Key Vault and advanced identity controls can be introduced in Sprint 2.

14. Sprint 1 Stretch Tasks

Only begin stretch work after the MVP and DevSecOps baseline are complete, tested and accepted.

Possible stretch tasks:

- Terraform modules
- Database migration/versioning
- Automated Azure deployment
- Enhanced SAST rules
- Enhanced DAST testing
- Data-governance checklist

Sprint 1 Expected Deliverables

Nationwide-TDP-2026/

```
│
├── README.md
├── docs/
│   ├── backlog.md
│   ├── DoR.md
│   ├── DoD.md
│   └── sprint1-reflection.md
├── src/
│   └── api/
├── tests/
│   └── api.tests/
├── infra/
│   ├── README.md
│   └── sql/
│       ├── schema.sql
│       └── seed.sql
└── .github/
    └── workflows/
        └── devsecops.yml
```

The exact structure can vary provided it supports the project requirements.

Sprint 1 Acceptance Criteria

Application

- Selected C# or Python 3.14 technology is documented
- API builds successfully
- /calculate endpoint exists
- Valid request returns HTTP 200
- Mock response is documented
- API contract is documented

Database

- Categories exists
- EmissionsFactors exists
- Assumptions exists
- Travel, Food and Utilities data is seeded
- T-SQL can retrieve the seeded data

DevSecOps

- Build runs automatically
- Unit tests run automatically
- SAST executes
- SCA executes
- DAST executes against the running API
- Findings are recorded
- Pipeline blocking behaviour is documented

Security

- No secrets committed to source control
- Synthetic/anonymised data is used
- Dependencies are identified
- Security evidence is retained

Agile

- Backlog is complete enough for Sprint 1
- Sprint 1 ownership is assigned
- DoR is defined
- DoD is defined
- Sprint reflection is completed
- Product Owner/Scrum Master sign-off is recorded

Sprint 1 End Showcase



Showcase emphasis

Demonstrate a **working technical foundation and DevSecOps workflow**, rather than the completed carbon engine.

The Sprint 1 showcase should demonstrate that the team can:

1. Commit your code to GitHub
2. Build the application
3. Run automated tests
4. Perform SAST
5. Perform SCA

6. Perform DAST against the running API
7. Call the API successfully
8. Demonstrate the seeded SQL data
9. Show security findings and remediation evidence

MVP delivery is the priority.

The Stretch objectives should only be undertaken once the MVP is complete, tested and signed off.